



INSTITUTO TECNOLÓGICO DE TEPIC
INGENIERÍA EN SISTEMAS COMPUTACIONALES

ACTIVIDAD 1 · UNIDAD 3

Operaciones CRUD en MongoDB

ALUMNO	Víctor Armando Camacho Correa
NÚMERO DE CONTROL	22400566
PROGRAMA ACADÉMICO	Ingeniería en Sistemas Computacionales
LUGAR Y FECHA	Tepic, Nayarit · Julio de 2026

1 Introducción

MongoDB es un sistema de gestión de bases de datos orientado a documentos. En esta actividad se construyó una base de datos temática llamada “spotify”, integrada por las colecciones “canciones” y “artistas”. A partir de ellas se practicaron las operaciones fundamentales de creación, lectura, actualización y eliminación de información mediante MongoDB Shell (mongosh).

El propósito del reporte es documentar cada instrucción ejecutada, mostrar su evidencia y explicar el resultado obtenido. También se incorporan correcciones de redacción y observaciones técnicas para que la entrega sea clara, ordenada y consistente.

1.1 Objetivo general

Aplicar operaciones CRUD en MongoDB mediante la creación de una base de datos musical, la inserción de documentos y la ejecución de consultas, actualizaciones y eliminaciones.

1.2 Objetivos específicos

- Crear la base de datos “spotify” y sus colecciones principales.
- Insertar documentos con información de canciones y artistas.
- Consultar información completa y aplicar filtros, proyecciones y ordenamientos.
- Actualizar campos específicos de los documentos existentes.
- Eliminar documentos, colecciones y la base de datos al finalizar la práctica.

1.3 Entorno de trabajo

mongosh Interfaz utilizada	spotify Base de datos	2 Colecciones	CRUD Operaciones
--------------------------------------	---------------------------------	-------------------------	----------------------------

1.4 Flujo general de la práctica

01 CREATE Crear e insertar	02 READ Consultar y filtrar	03 UPDATE Modificar campos	04 DELETE Eliminar datos
--	---	--	--

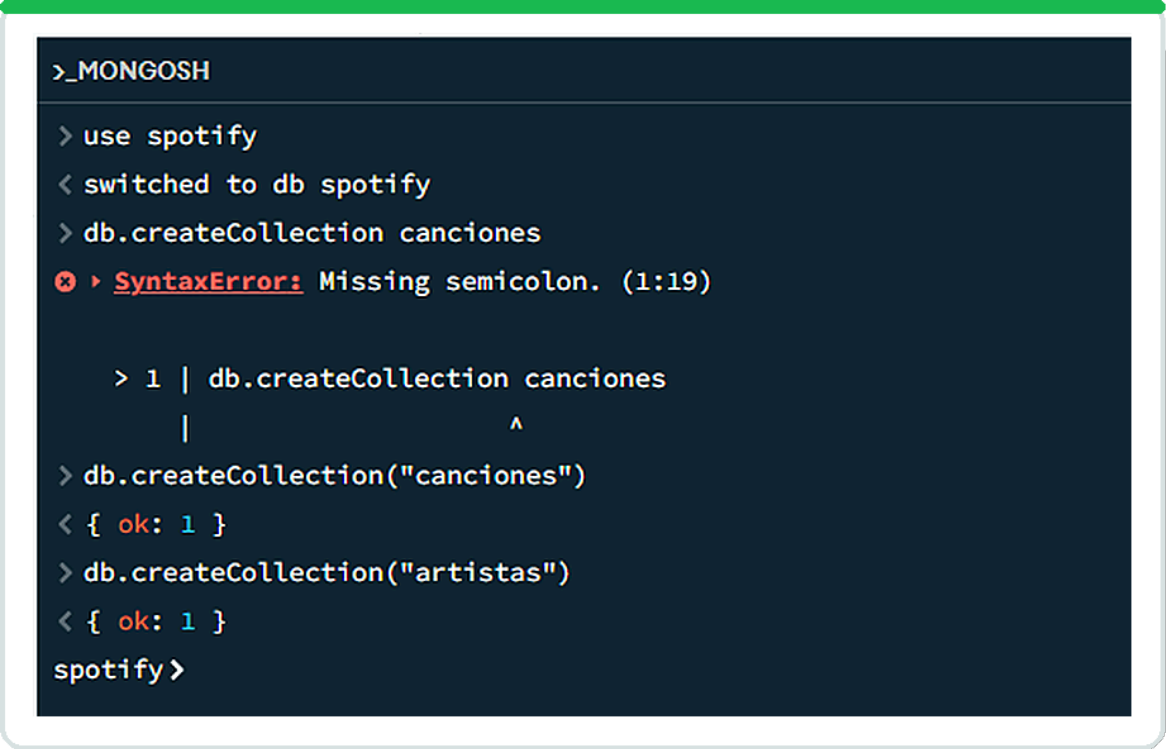
2 Desarrollo de la actividad

2.1 Creación de la base de datos y las colecciones

Primero se seleccionó la base de datos “spotify”. Posteriormente se crearon las colecciones “canciones” y “artistas”, las cuales almacenan documentos con estructuras independientes.

COMANDO UTILIZADO

```
use spotify
db.createCollection("canciones")
db.createCollection("artistas")
```



```
>_MONGOSH

> use spotify
< switched to db spotify
> db.createCollection canciones
❌ > SyntaxError: Missing semicolon. (1:19)

    > 1 | db.createCollection canciones
      |           ^
> db.createCollection("canciones")
< { ok: 1 }
> db.createCollection("artistas")
< { ok: 1 }
spotify>
```

Figura 1. Creación de la base de datos y de las colecciones.

La respuesta { ok: 1 } confirma que las colecciones fueron creadas correctamente.

Corrección aplicada: la primera instrucción mostró un error porque faltaban los paréntesis y las comillas. La sintaxis correcta es `db.createCollection("nombre")`.

2.2 Inserción de cinco canciones

Se utilizó insertMany() para registrar cinco documentos en una sola operación. Cada canción incluye título, artista, género, año, duración y número de reproducciones.

Título	Artista	Género	Año	Duración	Reproducciones
Time	Pink Floyd	Rock	1980	5	9,999,999
there was a time	Guns n Roses	Rock	1990	6	999,999
To Catch a thief	Lovage	Rock	1995	3	9,999
Touch	Daft Punk	Electro	2012	8	9,999,999
Dogs	Pink Floyd	rock	1990	16	999,999

OPERACIÓN PRINCIPAL

```
db.canciones.insertMany([
  { ... }, { ... }, { ... }, { ... }, { ... }
])
```

```
>_MONGOSH
>
> db["canciones"].insertMany([
  {
    titulo:"Time", artista:"Pink Floyd", genero:"Rock", anio:1980, duracion:5,reproducciones:9999999
  },{
    titulo:"there was a time", artista:"Guns n Roses", genero:"Rock", anio:1990,duracion:6,reproducciones:999999
  },{
    titulo:"To Catch a thief", artista:"Lovage",genero:"Rock",anio:1995,duracion:3,reproducciones:9999
  },{
    titulo:"Touch",artista:"Daft Punk",genero:"Electro",anio:2012,duracion:8,reproducciones:9999999
  },{
    titulo:"Dogs",artista:"Pink Floyd", genero:"rock",anio:1990,duracion:16,reproducciones:999999
  }
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a4d354d83b909c079c1dcce'),
    '1': ObjectId('6a4d354d83b909c079c1dccf'),
    '2': ObjectId('6a4d354d83b909c079c1dcd0'),
    '3': ObjectId('6a4d354d83b909c079c1dcd1'),
    '4': ObjectId('6a4d354d83b909c079c1dcd2')
  }
}
```

Figura 2. Inserción masiva de documentos en la colección "canciones".

El atributo acknowledged: true y los cinco identificadores generados confirman la inserción.

2.3 Inserción de cinco artistas

La colección “artistas” se llenó con cinco documentos. Los campos utilizados fueron nombre, país, género principal, edad y oyentes mensuales.

Nombre	País	Género principal	Edad	Oyentes mensuales
Daft Punk	Francia	French House	50	99,999
Juan Gabriel	México	Ranchera	66	99,999
Ozzy Osbourne	Inglaterra	Metal	76	99,999,999
kendrick Lamar	Estados Unidos	Hip Hop	39	9,999,999
Canserbero	Venezuela	Hip Hop	26	99,999

OPERACIÓN PRINCIPAL

```
db.artistas.insertMany([
  { ... }, { ... }, { ... }, { ... }, { ... }
])
```

```
> db["artistas"].insertMany([
  {
    nombre:"Daft Punk",pais:"Francia",generoPrincipal:"French House",edad:50,oyentesMensuales:99999
  },{
    nombre:"Juan Gabriel",pais:"Mexico",generoPrincipal:"Ranchera",edad:66,oyentesMensuales:99999
  },{
    nombre:"Ozzy Osbourne",pais:"Inglaterra",generoPrincipal:"Metal",edad:76,oyentesMensuales:99999999
  },{
    nombre:"kendrick Lamar",pais:"Estados Unidos",generoPrincipal:"Hip Hop",edad:39,oyentesMensuales:9999999
  },{
    nombre:"Canserbero", pais:"Venezuela",generoPrincipal:"Hip Hop",edad:26,oyentesMensuales:99999
  }
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a4d372e83b909c079c1dcd3'),
    '1': ObjectId('6a4d372e83b909c079c1dcd4'),
    '2': ObjectId('6a4d372e83b909c079c1dcd5'),
    '3': ObjectId('6a4d372e83b909c079c1dcd6'),
    '4': ObjectId('6a4d372e83b909c079c1dcd7')
  }
}
```

Figura 3. Inserción masiva de documentos en la colección “artistas”.
MongoDB asignó un ObjectId diferente a cada artista registrado.

2.4 Consulta de todos los documentos de “canciones”

La función `find()` sin filtros devuelve todos los documentos almacenados en una colección. La salida permite comprobar que los cinco registros conservan los campos definidos durante la inserción.

COMANDO UTILIZADO

```
db.canciones.find()
```

```
>_MONGOSH
> db["canciones"].find()
< {
  _id: ObjectId('6a4d354d83b909c079c1dcce'),
  titulo: 'Time',
  artista: 'Pink Floyd',
  genero: 'Rock',
  anio: 1980,
  duracion: 5,
  reproducciones: 9999999
}
{
  _id: ObjectId('6a4d354d83b909c079c1dccf'),
  titulo: 'there was a time',
  artista: 'Guns n Roses',
  genero: 'Rock',
  anio: 1990,
  duracion: 6,
  reproducciones: 999999
}
{
  _id: ObjectId('6a4d354d83b909c079c1dcd0'),
  titulo: 'To Catch a thief',
  artista: 'Lovage',
  genero: 'Rock',
  anio: 1995,
  duracion: 3,
  reproducciones: 9999
}
{
  _id: ObjectId('6a4d354d83b909c079c1dcd1'),
  titulo: 'Touch',
  artista: 'Daft Punk',
  genero: 'Electro',
  anio: 2012,
  duracion: 8,
  reproducciones: 9999999
}
{
  _id: ObjectId('6a4d354d83b909c079c1dcd2'),
  titulo: 'Dogs',
```

Figura 4. Consulta general de la colección “canciones”.

Se muestran los documentos con su ObjectId y los datos completos de cada canción.

2.5 Consulta de todos los documentos de “artistas”

La misma operación se aplicó a la colección “artistas”. Esto verifica la existencia de los cinco documentos y permite revisar sus atributos antes de realizar filtros o actualizaciones.

COMANDO UTILIZADO

```
db.artistas.find()
```

```
>_MONGOSH
> db["artistas"].find()
< {
  _id: ObjectId('6a4d372e83b909c079c1dcd3'),
  nombre: 'Daft Punk',
  país: 'Francia',
  generoPrincipal: 'French House',
  edad: 50,
  oyentesMensuales: 99999
}
{
  _id: ObjectId('6a4d372e83b909c079c1dcd4'),
  nombre: 'Juan Gabriel',
  país: 'Mexico',
  generoPrincipal: 'Ranchera',
  edad: 66,
  oyentesMensuales: 99999
}
{
  _id: ObjectId('6a4d372e83b909c079c1dcd5'),
  nombre: 'Ozzy Osbourne',
  país: 'Inglaterra',
  generoPrincipal: 'Metal',
  edad: 76,
  oyentesMensuales: 99999999
}
{
  _id: ObjectId('6a4d372e83b909c079c1dcd6'),
  nombre: 'Kendrick Lamar',
  país: 'Estados Unidos',
  generoPrincipal: 'Hip Hop',
  edad: 39,
  oyentesMensuales: 99999999
}
{
  _id: ObjectId('6a4d372e83b909c079c1dcd7'),
  nombre: 'Canserbero',
  país: 'Venezuela',
  generoPrincipal: 'Hip Hop',
  edad: 26,
  oyentesMensuales: 99999
}
```

Figura 5. Consulta general de la colección “artistas”.

La evidencia presenta nombre, país, género principal, edad y oyentes mensuales.

2.6 Consultas con condiciones

Los operadores de comparación permiten restringir los resultados. Para expresar “después de 2020” y “más de 10,000,000”, se utiliza el operador \$gt (greater than), ya que ambas condiciones son estrictas.

CANCIONES POSTERIORES A 2020

```
db.canciones.find({ anio: { $gt: 2020 } })
```

```
>_MONGOSH
> db["canciones"].find({anio:{$gte:2020}})
<
spotify> |
```

Figura 6. Búsqueda de canciones por año.

La consulta no devolvió documentos porque el registro más reciente de la colección corresponde al año 2012.

ARTISTAS CON MÁS DE 10,000,000 DE OYENTES

```
db.artistas.find({ oyentesMensuales: { $gt: 10000000 } })
```

```
>_MONGOSH
> db["canciones"].find({anio:{$gte:2020}})
<
> db["artistas"].find({oyentesMensuales:{$gte:10000000}})
< {
  _id: ObjectId('6a4d372e83b909c079c1dcd5'),
  nombre: 'Ozzy Osbourne',
  pais: 'Inglaterra',
  generoPrincipal: 'Metal',
  edad: 76,
  oyentesMensuales: 99999999
}
spotify>
```

Figura 7. Artistas que superan los 10,000,000 de oyentes mensuales.

El documento de Ozzy Osbourne cumple con la condición establecida.

Nota técnica: en la captura se observa \$gte, que significa “mayor o igual que”. Para respetar literalmente la palabra “después” o “más de”, el operador correcto es \$gt.

2.7 Proyección de campos

La proyección permite mostrar únicamente los campos necesarios. En este caso se ocultó el identificador `_id` y se conservaron el título y el artista de cada canción.

COMANDO UTILIZADO

```
db.canciones.find(  
  {},  
  { _id: 0, titulo: 1, artista: 1 }  
)
```

```
> db["canciones"].find({}, {_id:0,titulo:1,artista:1})  
< {  
  titulo: 'Time',  
  artista: 'Pink Floyd'  
}  
{  
  titulo: 'there was a time',  
  artista: 'Guns n Roses'  
}  
{  
  titulo: 'To Catch a thief',  
  artista: 'Lovage'  
}  
{  
  titulo: 'Touch',  
  artista: 'Daft Punk'  
}  
{  
  titulo: 'Dogs'
```

Figura 8. Proyección de los campos "título" y "artista".

El resultado reduce la salida y facilita la lectura de la información relevante.

2.8 Ordenamiento por año

El método `sort()` organiza los documentos de acuerdo con un campo. El valor `-1` indica orden descendente, por lo que las canciones aparecen desde la más reciente hasta la más antigua.

COMANDO UTILIZADO

```
db.canciones.find().sort({anio:-1})
```

```
>_MONGOSH
> db["canciones"].find().sort({anio:-1})
< {
  _id: ObjectId('6a4d354d83b909c079c1dcd1'),
  titulo: 'Touch',
  artista: 'Daft Punk',
  genero: 'Electro',
  anio: 2012,
  duracion: 8,
  reproducciones: 9999999
}
{
  _id: ObjectId('6a4d354d83b909c079c1dcd0'),
  titulo: 'To Catch a thief',
  artista: 'Lovage',
  genero: 'Rock',
  anio: 1995,
  duracion: 3,
  reproducciones: 9999
}
{
  _id: ObjectId('6a4d354d83b909c079c1dccf'),
  titulo: 'there was a time',
  artista: 'Guns n Roses',
  genero: 'Rock',
  anio: 1990,
  duracion: 6,
  reproducciones: 999999
}
{
  _id: ObjectId('6a4d354d83b909c079c1dcd2'),
  titulo: 'Dogs',
  artista: 'Pink Floyd',
  genero: 'rock',
  anio: 1990,
  duracion: 16,
  reproducciones: 999999
}
{
  _id: ObjectId('6a4d354d83b909c079c1dcce'),
  titulo: 'Time',
```

Figura 9. Canciones ordenadas por año de manera descendente.

La canción "Touch" aparece primero por tener el año más reciente del conjunto de datos.

2.9 Actualización de documentos

Para modificar un solo documento se empleó `updateOne()`. El primer argumento identifica el registro y el segundo contiene el operador `$set` con el campo que debe cambiar.

ACTUALIZAR REPRODUCCIONES

```
db.canciones.updateOne(
  { titulo: "Dogs" },
  { $set: { reproducciones: 999999999 } }
)
```

```
> db["canciones"].updateOne({titulo:"Dogs"},{$set:{reproducciones:999999999}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Figura 10. Actualización del número de reproducciones.

matchedCount: 1 y modifiedCount: 1 indican que el documento fue localizado y modificado.

ACTUALIZAR PAÍS

```
db.artistas.updateOne(
  { nombre: "Ozzy Osbourne" },
  { $set: { pais: "MUNDIAL" } }
)
```

```
> db["artistas"].updateOne({nombre:"Ozzy Osbourne"},{$set:{pais:"MUNDIAL"}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
spotify> |
```

Figura 11. Actualización del país de un artista.

matchedCount: 1 confirma la coincidencia. modifiedCount: 0 señala que el valor ya era igual al solicitado o que la operación se ejecutó previamente.

2.10 Eliminación, conteo y cierre de la práctica

Finalmente se eliminó una canción, se contó el número de documentos restantes y se borraron tanto la colección “artistas” como la base de datos “spotify”.

ELIMINAR UNA CANCIÓN

```
db.canciones.deleteOne({ titulo: "Time" })
```

```
> db["canciones"].deleteOne({titulo:"Time"})
< {
  acknowledged: true,
  deletedCount: 1
}
spotify> |
```

Figura 12. Eliminación del documento correspondiente a “Time”.

deletedCount: 1 confirma que se eliminó un documento.

Conteo final

```
db.canciones.countDocuments({})
```

```
> db["canciones"].countDocuments({})
< 4
spotify>
```

El resultado 4 confirma que queda un documento menos.

Eliminar colección

```
db.artistas.drop()
```

```
> db["artistas"].drop()
< true
spotify> |
```

El valor true indica que la colección fue eliminada.

Eliminar base de datos

```
db.dropDatabase()
```

```
> db.dropDatabase()
< { ok: 1, dropped: 'spotify' }
spotify>
```

MongoDB confirma que “spotify” fue eliminada.

Resultado final: la colección “canciones” quedó con 4 documentos después de eliminar “Time”. Posteriormente se eliminaron la colección “artistas” y la base de datos completa.

3 Resultados y conclusión

3.1 Síntesis de operaciones ejecutadas

Operación	Instrucción principal	Resultado
Seleccionar base de datos	use spotify	Contexto cambiado a spotify
Crear colecciones	db.createCollection()	2 colecciones creadas
Insertar documentos	insertMany()	5 canciones y 5 artistas
Consultar documentos	find()	Lectura completa de ambas colecciones
Filtrar información	\$gt	Consultas por año y oyentes
Proyectar campos	find({}, proyección)	Título y artista sin _id
Ordenar documentos	sort({ anio: -1 })	Año descendente
Actualizar documentos	updateOne() + \$set	Campos modificados
Eliminar documento	deleteOne()	1 canción eliminada
Contar documentos	countDocuments({})	4 canciones restantes
Eliminar estructura	drop() / dropDatabase()	Colección y base eliminadas

3.2 Observaciones de calidad

Consistencia de nombres: conviene unificar mayúsculas y minúsculas en valores como “Rock/rock” y “kendrick Lamar/Kendrick Lamar”.

Calidad de los títulos: el valor “To Catch a thief” puede corregirse antes de una entrega definitiva para mantener una nomenclatura uniforme.

Elección de operadores: cuando la condición es estricta, debe emplearse \$gt; \$gte se reserva para “mayor o igual que”.

Diseño de datos: en un proyecto real es preferible almacenar fecha de nacimiento en lugar de edad, ya que la edad cambia con el tiempo.

3.3 Conclusión

La práctica permitió recorrer de forma completa el ciclo CRUD en MongoDB. Se creó una base de datos, se almacenaron documentos con diferentes campos, se realizaron consultas generales y filtradas, se controló la información mostrada mediante proyecciones y se aplicó un ordenamiento descendente. Además, se actualizaron registros específicos y se verificaron los indicadores de coincidencia y modificación proporcionados por MongoDB.

Finalmente, se eliminaron documentos y estructuras de datos de forma controlada. Con ello se comprobó que MongoDB ofrece una sintaxis flexible y directa para trabajar con colecciones orientadas a documentos. La actividad también evidenció la importancia de mantener datos consistentes, utilizar operadores adecuados y documentar cada resultado para facilitar su validación.

Aprendizaje logrado: se aplicaron correctamente las operaciones CREATE, READ, UPDATE y DELETE, así como filtros, proyecciones, ordenamientos y conteos.